

Backtest Integrity Auditor (BIA): deflate the numbers, read the code that produced them

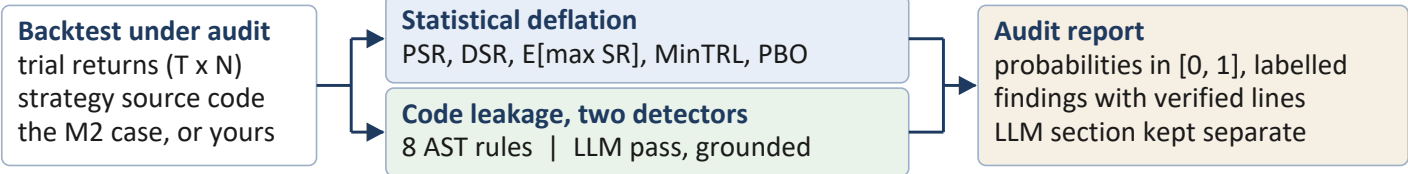
Théo Crochemar, M.Sc. Applied Mathematics & Quantitative Finance, Université Paris 1 Panthéon-Sorbonne.
Polymer Tech Expo 2026. Python, 288 tests, GitHub: tcroche.

Problem statement

A backtest can be wrong in two independent ways. Statistically: when many configurations are tried and the best is reported, the in-sample Sharpe ratio is inflated by selection, a bias Bailey and López de Prado quantified (Deflated Sharpe Ratio, Probability of Backtest Overfitting) but that few pipelines compute. In code: look-ahead leaks such as a negative shift, a centred window or a scaler fitted before the split feed the future into the signal, and finding them is manual review. My own M.Sc. systematic-trading project showed both: an in-sample Sharpe of 1.9 built on 18 tuned configurations, and a write-up in which I had written that it “collapsed to a deflated 0.92”, treating a DSR, a probability, as a Sharpe ratio. The tool exists to catch exactly that, in the numbers and in the prose.

Solution overview

BIA is a Python tool that audits a backtest on two axes and writes one report. The statistical axis takes the matrix of trial returns and computes PSR, E[max SR], DSR, MinTRL and CSCV/PBO with its simulated null, plus concentration diagnostics: share of P&L in the best day, Sharpe without it, correlation between trials. The code axis scans the strategy source with eight deterministic AST rules (exact lines), then a language-model pass for what syntax cannot express, such as a same-day aggregate broadcast to every intraday bar. The two detectors are never merged. On my M2 case the report reproduces the original DSR (0.929 with the Russell 2000, 0.378 without), finds 70% of the in-sample profit in a single session (2025-04-09) with 18 of 18 grid cells drawing more than half their P&L from that day, and no leak in the code (0 rule detections; the semantic pass raised only questions, each resolved by the engine's own convention): the code was clean, the leak was in the selection.



Use of AI

AI plays two roles, kept apart. **As a component:** Claude (claude-sonnet-4-6, Anthropic API, temperature 0, JSON only) performs the semantic code pass. The prompt lives in a versioned YAML registry with a changelog (v1.0.0 to v1.2.0). Its system prompt says, in short: “Report only leakage that no syntactic rule can express. Every finding must cite line numbers that exist. Every finding carries an external_dependency field: null only if the leak is established within this file alone; otherwise name the external fact to check, set severity to review, and make the fix conditional on it.” The harness trusts none of it: line numbers are checked against the file, snippets come from the real source, and severity is capped at “review” in code unless the model declares the leak self-contained. Responses are cached by SHA-256 of the exact prompt pair. **As a coding assistant:** Claude in the chat interface produced scaffolds, first implementations and tests; I checked every formula against the papers and kept a dated DEVLOG of prompts, iterations and mistakes.

Impact & value

For a multi-manager platform, BIA is a pre-mortem on any strategy submission: one command answers how many trials were run, how concentrated the P&L is and whether the code sees the future. Measured on a seeded benchmark of 16 files (11 leaks, 6 controls), plus the tool's own source and my real strategy code audited live in the same run:

Detector	Syntactic leaks (9)	Semantic leaks (2)	FP on 6 control files
AST rules	9 / 9	0 / 2	0
LLM pass (prompt v1.2.0)	7 / 9	2 / 2	0
Combined	9 / 9	2 / 2	0

0 invented line numbers over 16 files; 0 findings on the tool's 28 files and the 16 M2 files. Each half catches what the other cannot: that complementarity, not round numbers on a small benchmark we wrote ourselves, is the result.

Reflections

Three lessons, each with a scar. **A prompt rule is a request, code is a guarantee:** prompt v1.1.0 asked the model to cap cross-file-conditional findings at “review”; on real code it returned “medium” and a fix that would have delayed a causal signal, so the cap moved into the harness. **Negative instructions are weakly obeyed:** the model re-reported 6 of 8 patterns it was told to leave to the rules; the deterministic layer is the floor. **Benchmarks audit their author:** seeding trapped files exposed a precision gap in one rule and naming artefacts in my own cases. Challenges: unit discipline (DSR is a probability, and a guard raises on the mix-up), reproducing published numbers to the digit, and a PBO whose own null has standard deviation 0.21 at T = 103. Next: a UI layer, validation on third-party code with independently known leaks, and run-to-run variance of the semantic pass.